Informe del Trabajo Obligatorio

Sistema de gestión de torneo de fútbol 5

Matías Fadel, Gabriel Mederos, Diego Costa

06/11/2026

Fundamentación e Interpretación

El objetivo de esta actividad es reemplazar la manera en la que se organiza y mantiene registro de un torneo interfacultad de fútbol 5 por un sistema unificado que permita cumplir las mismas funciones de manera más sencilla, eficiente y segura.

A la hora de diseñar se tomaron en cuenta principalmente evitar los posibles errores a la hora de ingresar datos que se generan al usar el sistema actual. Las principales decisiones tomadas se relacionan con la base de datos, decidimos usar triggers, chequeos automáticos y autenticaciones directamente en el código del SQL por lo que resultó quedando relativamente largo, por lo cual decidimos separarlo y agruparlos en archivos según su función, como por ejemplo la creación de las tablas, los datos maestros, las vistas y los triggers. Nuestra idea en base a la consigna era incluir chequeos constantes en las distintas capas (back-end y base de datos) para evitar lo más posible esos mismos errores que existen con el sistema el día de hoy. No desarrollamos un front-end, ya que la letra lo deja como opcional; la aplicación se utiliza a través de su API. La organización de archivos de la base de datos se puede ver aquí, con una explicación por arriba de la función de cada uno:

- **01 - Tablas y checks**: Creación de la base con sus tablas y restricciones (es decir, checks).
- **02 - Datos maestros**: Turnos, facultades, complejos, canchas y usuarios iniciales de la aplicación.
- **04 - Vistas**: Las vistas que implementan los reportes.
- **05 - Triggers**: Función de suspensión, procedimientos de validación y triggers.
- **06 - Procedimientos**: Operaciones que usa la aplicación (planilla, goles, tarjetas, partidos, sanciones, estados).
- **07 - Datos de prueba**: Su nombre se explica a sí mismo.
- **08 - Usuarios**: Usuarios de MySQL con permisos diferenciados según su rol.

Aparte, en la carpeta *tests*:

- **09 - Casos que deben fallar**: Lo contrario a los datos de prueba, lo que hay aquí debe fallar.

Al momento de comenzar a trabajar nos encontramos con variados enunciados en los que se podían tener más de una interpretación, por lo tanto, decidimos primero de todo interpretarlos para aclarar cualquier clase de confusión (tanto por nuestra parte o en caso de que alguno de los profesores entendiera de distinta manera la letra). Por ejemplo, se menciona que la suspensión por

tarjetas amarillas se da cuando se alcanzan 3 en cada edición. Nosotros entendemos que puede haber más de una suspensión, por lo que suceden cuando llegan a un múltiplo de 3 (3,6,9…). Además, las dobles amarillas en los partidos también cuentan para este contador, por más que equivalgan a una roja.

Mejoras

Se han hecho algunos cambios en comparación al modelo original planteado en la letra del ejercicio:

- Se creó la tabla *fecha_torneo* porque la letra agrupa los partidos en fechas, pero no establece cuándo ocurre cada fecha. De esta manera se puede validar la información sobre el partido.
- Se establece que la clave primaria de *partido* sea solo *id_partido* porque esta misma es autoincremental por sí misma y vuelve redundante a id_torneo. Si el partido pertenece al torneo se puede validar utilizando las claves foráneas.
- Se creó el atributo *partido.fecha_hora_jugado* para marcar el momento de finalización del partido con el objetivo de permitir hacer modificaciones hasta 48hs luego de que el partido fue marcado como jugado.
- Se crea también el atributo *sancion_jugador.id_tarjeta_origen* el cual permite trazar la sanción a su tarjeta en caso de que una fuera asignada por error. Es decir, que quitar tarjetas puestas por error también quitaría la sanción asociada a ella.
- El resultado no se almacena, sino que se calcula basado en los goles, es decir, que la tabla de posiciones nunca se desincroniza.

Algunas otras mejoras que incluimos que no son tan específicas incluyen que por ejemplo, las reglas y datos se encuentren tanto en el backend y la base de datos para una mayor cantidad de filtro de datos y un archivo con casos de prueba negativos, para poder demostrar que los casos donde deberían saltar alarmas efectivamente sucede.

Finalmente decidimos utilizar Docker para la comodidad del usuario, ya que reduce los problemas posibles de compatibilidad o de tener diferentes versiones instaladas, además de que con un comando levanta la base de datos de manera fácil.

Bitácora

28 / 10 / 2026 -

Este día comenzamos a trabajar en el proyecto. Decidimos leer la letra y comenzar a plantear el trabajo y todas las características que este mismo deberá de tener y cumplir para poder luego organizar la carga de trabajo y de las diferentes partes que el mismo conlleva de manera eficiente.

Iniciamos con la parte a realizar en SQL, analizando lo que la letra decía, puntos a tener en cuenta, restricciones y posibles maneras de mejorar y/o optimizar el código y su organización.

30 / 10 / 2026 -

Para tener una idea de cómo poder optimizar el programa y simplificar/acortar lo que debíamos ejecutar le preguntamos al chatbot de Claude algunas posibilidades. Un concepto que mencionó fue el de triggers. Los triggers son básicamente comandos que se ejecutan por su cuenta cuando alguna persona hace un comando que modifique una tabla (es decir, un update, insert o delete) y permite decidir qué hacer en base a lo que quiere modificar, es decir, permite de antemano decirle a la base de datos qué hacer en tales casos. Por ejemplo, si se quiere borrar un partido es posible utilizar un trigger para chequear que el partido no se haya jugado aún.

Esto nos pareció una idea interesante y muy útil para el problema que nos fue planteado y decidimos seguir con ella.

03 / 11 / 2026 -

Este día comenzamos a trabajar de lleno en el código del obligatorio, especialmente en SQL. Definimos una organización dentro del repositorio y separamos lo más posible las secciones para tenerlo organizado y entendible. Establecimos y creamos la base de datos, sus tablas, chequeos y distintos triggers. También, le dimos la consigna del entregable y el código realizado a Claude para que nos generara datos de prueba adecuados que permitan cumplir con la consigna, ya que hacerlo a mano hubiera llevado mucho tiempo y sería muy ineficiente.

03 / 11 / 2026 hasta 05 / 11 / 2026 -

A lo largo de la semana continuamos esporádicamente progresando tanto en finalizar detalles de la base de datos como en el backend a medida que comenzamos a definir la información requerida en el informe.

Respecto a la base de datos le añadimos casos que deben fallar a la hora de ejecutarse para complementar el trabajo, además de añadir los usuarios “planillero” y “administrador” junto a sus

permisos a la hora de acceder y manipular la base de datos.

06 / 11 / 2026 hasta 08 / 11 / 2026 -

Con una extensión hasta el domingo acordada decidimos tomarnos el tiempo para ultimar algunos detalles como el informe, el README y algunas cuestiones de la organización del repositorio para prepararlo para una entrega organizada. También integramos a Docker como herramienta para la entrega del trabajo.

Bibliografía

Anthropic. (2026). Claude (versión Opus 5.5) [Modelo de lenguaje de gran tamaño]. https://claude.ai

Oracle Corporation. (2026). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

Pallets Projects. (2026). Flask Documentation. https://flask.palletsprojects.com/

Python Cryptographic Authority. (2026). bcrypt. https://pypi.org/project/bcrypt/

Docker Inc. (2026). Docker Compose Documentation. https://docs.docker.com/compose/

Utilizamos Claude AI para llevar a cabo algunas secciones del trabajo (por ejemplo, crear datos de prueba y como fuente de información para lograr entender temas nuevos como triggers o integrar la entrega en Docker). Entendemos esto como un uso razonable de la IA como herramienta para hacer más eficiente el trabajo y como una herramienta de aprendizaje.